UNIVERSITÀ DEGLI STUDI DI CATANIA

Dipartimento di Matematica e Informatica

Corso di Laurea Magistrale in Informatica

Emanuel Ramaci

Smart Household Advisor: Decision Support System for Integrated Household Expense and Energy Analysis

Relazione Progetto di Ingegneria dei Sistemi Distribuiti

Professore: Emiliano Alessio Tramontana

Anno Accademico 2025 - 2026

Indice

1 Introduzione **2**
1.1 Contesto e motivazione 2
1.2 Obiettivi del sistema 2

2 Requisiti **3**
2.1 Requisiti funzionali 3
2.2 Requisiti non funzionali 5

3 Architettura e scelte progettali **6**
3.1 Moduli principali 6
3.2 API e sicurezza 7
3.3 Design pattern e scelte progettuali 7

4 Simulazioni e gestione dei dati energetici **9**
4.1 Tipologie di simulazioni 9
4.2 Parametri e risultati 9

Capitolo 1

Introduzione

1.1 Contesto e motivazione

La gestione delle spese domestiche e dei consumi energetici rappresenta oggi una sfida rilevante sia dal punto di vista economico che ambientale. L'aumento dei costi dell'energia e la complessità delle tariffe rendono difficile comprendere con precisione dove si concentrino le maggiori spese e quali comportamenti incidano in modo significativo sul budget familiare.

Gli strumenti tradizionali non offrono una visione integrata tra spese ed energia, né consentono di valutare in modo immediato l'impatto di possibili decisioni future. Da questa esigenza nasce **Smart Household Advisor**[1], un sistema in grado di analizzare i dati storici e supportare l'utente nella gestione consapevole delle proprie risorse domestiche.

1.2 Obiettivi del sistema

Il progetto si propone di realizzare una piattaforma di supporto alle decisioni per l'analisi integrata di spese e consumi energetici domestici. Il sistema consente di elaborare dati storici suddivisi per categoria, individuare trend e fornire report sintetici tramite **API** dedicate.

Inoltre, integra funzionalità di simulazione "what-if" per stimare il risparmio ottenibile attraverso specifiche scelte operative. Dal punto di vista progettuale, l'obiettivo è applicare principi di modularità, separazione delle responsabilità e meccanismi di sicurezza basati su autenticazione **JWT** (JSON Web Token) e controllo degli accessi per ruolo (**RBAC**).

[1] Il codice sorgente del progetto è disponibile al seguente repository GitHub: `https://github.com/Emanuel-Ramaci/Smart-Household-Advisor`.

Capitolo 2

Requisiti

2.1 Requisiti funzionali

Il sistema deve consentire, attraverso le pagine dedicate mostrate in Figura 2.1, la registrazione di nuovi utenti e il successivo accesso tramite credenziali memorizzate nel database.

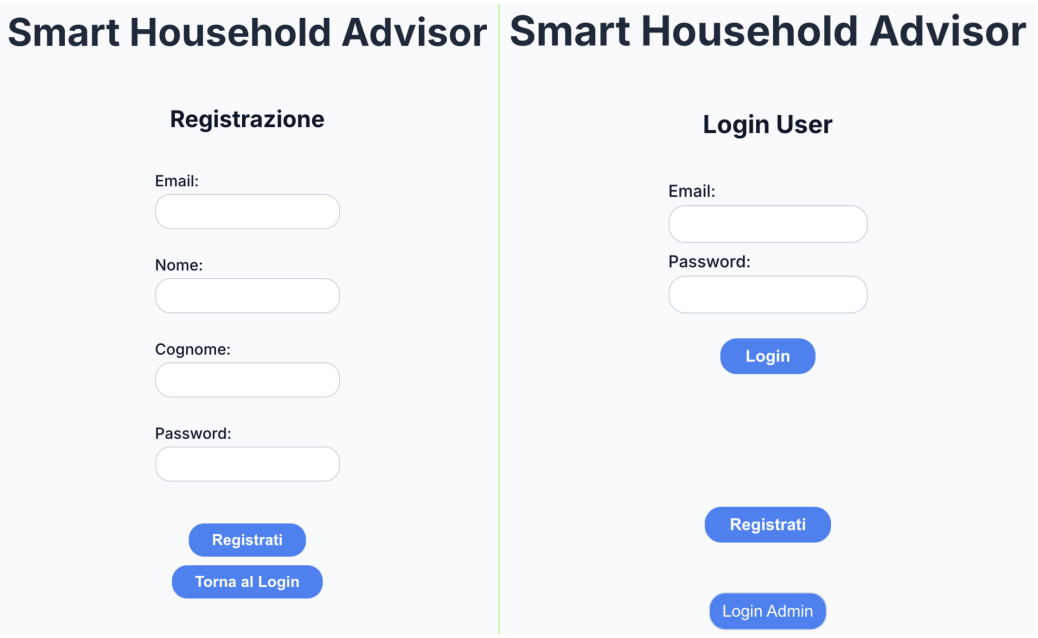

Figura 2.1: Pagine di registrazione e login utente

Il sistema deve supportare **operazioni CRUD** su spese, consumi e informazioni della household, garantendo che l'accesso sia consentito solo agli

utenti effettivamente associati alla casa. Il proprietario di una household può inoltre modificarla o eliminarla.

Sono previste funzionalità di analisi dei consumi per categoria e la possibilità di eseguire tre tipologie di simulazione: Heating, Time Shift e Standby, al fine di stimare potenziali risparmi.

Dal punto di vista dei ruoli, un utente **ADMIN**, tramite la dashboard dedicata mostrata in Figura 2.2, può visualizzare l'elenco degli utenti registrati, aggiungerne di nuovi, attivarli, disattivarli o eliminarli. Può inoltre gestire tutte le altre risorse del sistema, come le household e i risultati delle simulazioni. Un utente **USER** invece, tramite la home in Figura 2.3, può gestire solo le household a cui appartiene: eseguire simulazioni su queste, crearne di nuove (diventandone proprietario di default), aggiungervi spese e consumi elettrici.

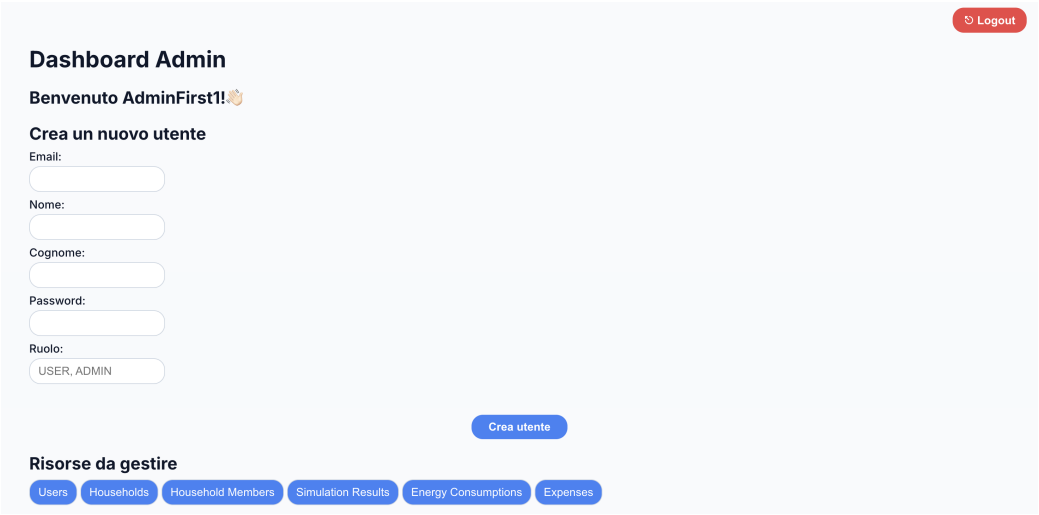

Figura 2.2: Dashboard web dedicata agli utenti ADMIN

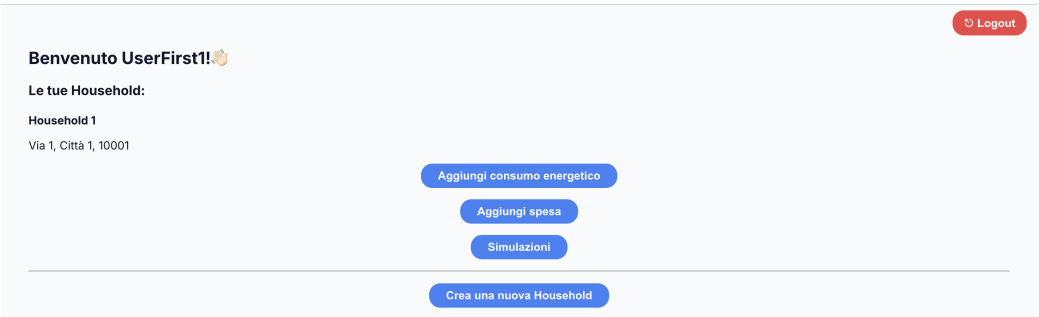

Figura 2.3: Home web dedicata agli utenti USER

2.2 Requisiti non funzionali

Il sistema deve garantire adeguati livelli di sicurezza. Le password degli utenti non vengono mai memorizzate in chiaro, ma salvate come hash tramite l'utilizzo della libreria **SCryptUtil**; in fase di login viene effettuato il confronto tra hash tramite verifica crittografica. L'autenticazione avviene tramite token JWT `(energy_app/security)`, che devono essere validi, non scaduti e presenti in un **TokenStore** per consentire l'accesso alle risorse. Il controllo degli accessi è basato su RBAC (Role-Based Access Control), con ruoli distinti per ADMIN e USER e relativi permessi associati.
La configurazione dei permessi e dei ruoli è definita in `energy_app/runner/DataInitializer.java`.

Dal punto di vista architetturale, il sistema deve garantire modularità e separazione delle responsabilità. Le simulazioni sono implementate tramite il pattern **Strategy** `(energy_app/service/simulation/strategy)`, mentre l'accesso alle funzionalità avviene attraverso un **Remote Facade** `(energy_app/service/simulation/remote_facade)` che delega le richieste ai componenti appropriati. L'accesso alle risorse sensibili è protetto tramite un **Protection Proxy** `(energy_app/service/simulation/proxy/protection)`, che verifica i permessi e l'appartenenza alla household, e un **Cache Proxy** `(energy_app/service/simulation/proxy/cache)`, che gestisce la memorizzazione temporanea dei risultati di simulazione per migliorare l'efficienza.

Infine, il sistema deve garantire coerenza nello stato di autenticazione: al logout o alla chiusura della sessione, il token dell'utente viene rimosso dal TokenStore, rendendo impossibile qualsiasi operazione successiva con token invalidati.

Capitolo 3

Architettura e scelte progettali

Il sistema è stato sviluppato come un monolite modulare, organizzato in diversi moduli che separano chiaramente i livelli di presentazione, logica di business e persistenza dei dati. Il backend è sviluppato con **Spring Boot**, mentre il database utilizzato è **MySQL**, configurato tramite il file `application.properties` del progetto. Per scopi esclusivamente di sviluppo, le credenziali dell'utente del database sono state impostate su `root/root`.

3.1 Moduli principali

Nel package `energy_app/model` sono definiti i principali moduli del sistema:

- `User, Role, Permission`: gestione degli utenti, dei ruoli e dei permessi per implementare il controllo degli accessi basato su RBAC;
- `Household, HouseholdMember`: rappresentano le abitazioni e i membri associati, permettendo di tracciare proprietà, appartenenza e ruoli all'interno della casa;
- `EnergyConsumption, Expense`: raccolta e gestione dei dati relativi a consumi energetici e spese generiche della household;
- `SimulationResult`: memorizza i risultati delle simulazioni effettuate dagli utenti.

3.2 API e sicurezza

Le funzionalità del sistema sono esposte tramite API coarse grained, che aggregano più informazioni in una singola chiamata per semplificare l'integrazione con la dashboard web. L'autenticazione avviene tramite token JWT, memorizzati in un TokenStore, e l'accesso alle risorse è limitato agli utenti autorizzati secondo il proprio ruolo e la membership della household.

3.3 Design pattern e scelte progettuali

Per garantire modularità e manutenibilità, sono stati integrati diversi design pattern tra cui:

- **Remote Facade**: unifica l'accesso ai servizi di simulazione, nascondendo la complessità della logica sottostante. L'implementazione è collocata nel package `energy_app/service/simulation/remote_facade`;
- **DTO (Data Transfer Object)**: utilizzati per il trasferimento dei dati tra client e server, sia per gli utenti (profilo, cambio password, informazioni amministrative) sia per le household, spese, consumi energetici e risultati di simulazione. I relativi file sono definiti nel package `energy_app/dto`;
- **Authenticator**: verifica le credenziali degli utenti confrontando la password in input, elaborata con Scrypt, con l'hash memorizzato nel database. Il servizio di autenticazione è implementato nel file `energy_app/service/AuthService.java`.

L'**AOP** (Aspect Oriented Programming) è stata invece applicata per il logging centralizzato, monitorando tutti i tentativi di simulazione tramite l'interfaccia `ISimulationService` e l'accesso ai risultati tramite `ISimulationResultService`. Eventuali tentativi di accesso non autorizzati vengono registrati. Il funzionamento dell'AOP è illustrato in Figura 3.1. L'implementazione è definita nel package `energy_app/aspect`.

Questa architettura modulare consente di mantenere separati i livelli, semplifica la manutenzione e garantisce sicurezza e tracciabilità delle operazioni critiche.

```
[Aspect] Tentativo di simulazione - type: HEATING, householdId: 1
Subject: user1@example.com
Role claim: USER
```

```
[Aspect] Tentativo di simulazione - type: HEATING, householdId: 1
[CacheProxy] Cache miss
[Aspect] Tentativo di simulazione - type: HEATING, householdId: 1
```

```
[Aspect] Tentativo di simulazione - type: HEATING, householdId: 1
[CacheProxy] Cache miss
[Aspect] Tentativo di simulazione - type: HEATING, householdId: 1
Hibernate:
    insert
    into
        simulation_results
        (created_at, estimated_saving_euro, estimated_saving_kwh, h
    values
        (?, ?, ?, ?, ?, ?, ?)
[Aspect] SUCCESSO - completato in 4 ms
[Aspect] SUCCESSO - completato in 6 ms
[Aspect] SUCCESSO - completato in 18 ms
```

```
[Aspect] Utente 1 tenta di accedere ai risultati simulazione - householdId: 1
Subject: user1@example.com
Role claim: USER
```

```
[Aspect] Utente 1 tenta di accedere ai risultati simulazione - householdId: 1
[CacheProxy] Cache miss per household 1
Subject: user1@example.com
Role claim: USER
```

```
[Aspect] Utente 1 tenta di accedere ai risultati simulazione - householdId: 1
Hibernate:
    select
        sr1_0.id,
        sr1_0.created_at,
        sr1_0.estimated_saving_euro,
        sr1_0.estimated_saving_kwh,
        sr1_0.household_id,
        sr1_0.input_data,
        sr1_0.result_data,
        sr1_0.simulation_type
    from
        simulation_results sr1_0
    where
        sr1_0.household_id=?
[Aspect] SUCCESSO - metodo completato in 4 ms
[Aspect] SUCCESSO - metodo completato in 7 ms
[Aspect] SUCCESSO - metodo completato in 22 ms
```

Figura 3.1: Aspect Oriented Programming per simulazioni e accesso ai risultati

Capitolo 4

Simulazioni e gestione dei dati energetici

Il sistema consente agli utenti di eseguire simulazioni per valutare l'impatto di diverse strategie sui consumi energetici e sulla bolletta della propria abitazione.

4.1 Tipologie di simulazioni

Il sistema sviluppato permette di effettuare tre tipologie di simulazioni, implementate tramite il design pattern Strategy nel package `energy_app/service/simulation/strategy`:

1. **Heating**: valuta l'effetto della regolazione della temperatura dei sistemi di riscaldamento, considerando il consumo attuale e la riduzione di temperatura;

2. **Time Shift**: pianifica l'uso dei dispositivi nelle fasce orarie a minor costo, spostando i consumi dalle ore di punta (dove il costo dell'energia è maggiore) a quelle fuori punta (dove il costo è minore);

3. **Standby**: stima il risparmio ottenibile spegnendo o scollegando i dispositivi in standby, riducendo consumi invisibili ma significativi nel tempo.

4.2 Parametri e risultati

Le simulazioni considerano i seguenti parametri di input:

- `currentConsumptionKwh`: consumo attuale in kWh;
- `energyTariffEuro`: tariffa energetica misurata in €/kWh;
- `reductionDegrees`: riduzione della temperatura per il riscaldamento;
- `standbyConsumption`: consumi in standby dei piccoli dispositivi;
- `applianceConsumption`: consumi di apparecchi elettrici ad alto consumo, come fan coil o stufe elettriche.

A partire da questi dati, dopo aver effettuato la simulazione richiesta dall'utente, il sistema produce i seguenti risultati:

- `simulationType`: tipo di simulazione eseguita (Heating, Standby, Time Shift);
- `estimatedSavingEuro`: risparmio stimato in euro;
- `estimatedSavingKwh`: risparmio stimato in kWh;
- `resultData`: dettagli dei calcoli relativi allo scenario simulato.

In Figura 4.1 è riportato un esempio di simulazione di tipo Heating Reduction.

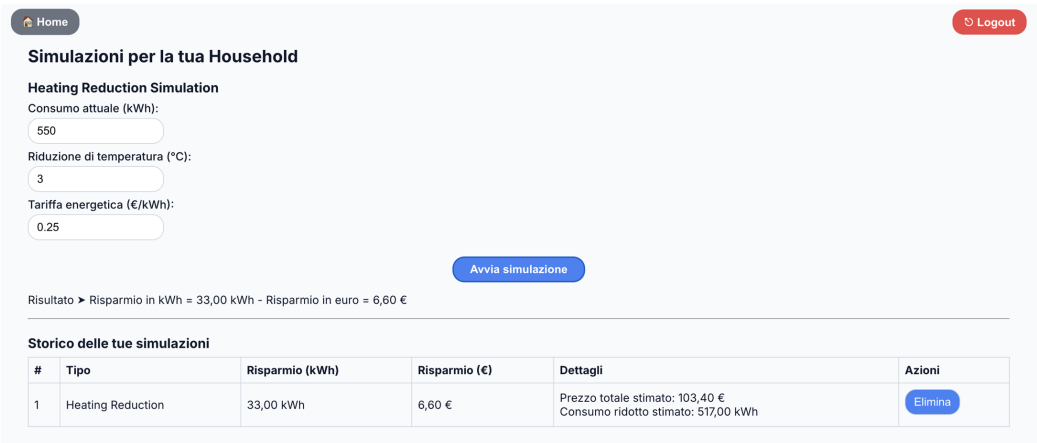

Figura 4.1: Esempio di simulazione Heating e risultato registrato dal sistema

Grazie a queste simulazioni, l'utente può confrontare scenari alternativi, pianificare l'uso dei dispositivi nelle fasce orarie più convenienti e prendere decisioni informate per ridurre i consumi e ottimizzare la spesa energetica,

riducendo così l'importo della bolletta. Tutti i risultati delle simulazioni vengono salvati e possono essere consultabili successivamente, garantendo trasparenza e tracciabilità delle analisi effettuate.